End-to-end Machine Learning Report

-- Prediction on Mortgage Loan Application

Team 23: Veronica Joe (jj3470), Crystal Guo (lg3434), Shuxuan Xu (sx2412), Fangyi Lin (fl2748)

Github Repository: https://github.com/xsxsx-999/5243-project4

ShinyApp: https://crystalguo.shinyapps.io/mortgage-approval-estimator/

1. Project Overview

1.1 Introduction

The objective of this project is to build an end-to-end machine learning pipeline to decode loan decisions within the United States mortgage market. Using data from the Home Mortgage Disclosure Act (HMDA), we aim to perform a comprehensive analysis of the factors driving application outcomes. By engineering robust features and training predictive models, this study seeks to identify the key drivers—ranging from applicant demographics to loan characteristics—that influence whether a mortgage application is ultimately approved or denied.

1.2 Data Collection

To construct a focused and computationally viable dataset, our data acquisition and sampling strategy followed a strict, predefined methodology:

- **Data Source & Geographical Scope:** We obtained historical mortgage application records from the HMDA database spanning a six-year period from **2019 to 2024**. To capture representative trends while managing data volume, the geographical scope was restricted to two of the largest and most dynamic housing markets in the U.S.: **California (CA) and Texas (TX)**.

- **Target Formulation & Filtering:** The full HMDA dataset contains multiple stages of the application lifecycle (e.g., withdrawn, incomplete). For the purpose of binary classification, we isolated records to strictly two definitive outcomes.**Originated** is mapped to `target_y = 1`, while **Denied** is mapped to `target_y = 0`

- **Stratified Sampling:** Given the massive scale of the raw dataset, we applied a **1% stratified proportional sampling fraction** strictly within the restricted two-class subset, which guarantees that the *class proportion* of the Originated vs. Denied subset is perfectly preserved. This yielded a final analytical sample of **139,991** records.

1.3 Statistical Interpretation & Caveats

Our models explicitly estimate the **conditional probability** of origination:

$$P(\text{Originated} \mid \text{Originated or Denied})$$

They do *not* estimate the unconditional probability $P(\text{Originated})$. Therefore, all subsequent metrics, feature importance, and fairness analyses apply to **applications with a definitive resolution**.

2. Cleaning & Preprocessing the data

To ensure data integrity and prevent data leakage, our preprocessing pipeline computes all **data-driven** transformations strictly on the **training set**, then applies the learned parameters to the test set.

2.1 Datatype Formatting & Feature Engineering

To prevent data leakage, we first removed post-application variables (e.g., `action_taken`, `denial_reason`) and enforced strict numeric and string type casting while preserving missingness. We then engineered two key variables: transforming `activity_year` into a categorical `covid_phase` to capture macroeconomic shifts, and parsing HMDA's categorical DTI strings into an ordinal numeric feature (`debt_to_income_ratio_ord`) coupled with a binary missingness indicator (`dti_missing_flag`).

2.2 Drop Columns

To reduce noise, prevent the curse of dimensionality, and optimize computational efficiency, features were systematically dropped based on missingness, variance, and cardinality thresholds:

2.2.1 High Missingness

Any column with a missing value rate exceeding **80%** was directly dropped, as imputing such heavily sparse features would introduce severe artificial bias.

2.2.2 Near-Zero Variance

Features lacking sufficient variation provide no predictive signal. For continuous columns, we dropped those with a standard deviation ≤ 0.03. For categorical columns, features were dropped if a single dominant category accounted for $\geq 98\%$ of the observations.

2.2.3 High Cardinality & Tiered Filtering

To prevent sparse matrix explosion during One-Hot Encoding, we applied a tiered cardinality filter to categorical features. Columns with **> 8 unique values** were dropped to minimize noise, while those with **5–8 unique values** were flagged for manual consolidation (e.g., bucketing rare categories to improve robustness; see Appendix-1).

2.3 Imputation

Missingness patterns were first analyzed via heatmap (Appendix-2) , followed by a seires of imputation.

1. **Numeric Imputation**: We applied **KNNImputer (k=5)** to missing numerics, utilizing related predictors for context. *(Features were temporarily scaled to prevent large magnitudes from dominating the distance metric)*.
2. **Categorical Imputation**: Applied standard mode imputation.
3. **Discrete Median Override**: `loan_term` and `debt_to_income_ratio_ord` were explicitly forced to median imputation to maintain interpretability.

2.4 Log1p Transformation

To stabilize variance and reduce the impact of extreme right-skew in monetary fields, we applied a **log1p transformation** ($y = \log(1 + x)$). Negative values were safely clipped to 0 prior to transformation.

- **Fixed Application**: Unconditionally applied to strictly positive, heavy-tail features: `loan_amount` and `property_value`.
- **Dynamic Application**: Applied to `income` *only if* its training distribution exhibited strong right-skewness (skew > 1.0, $N \geq 100$).

2.5 Outliers (Winsorization)

To mitigate the impact of anomalies, we applied **Winsorization** by clipping values at the 1st and 99th percentiles. Boundaries were derived strictly from the training set and applied to both splits. This process was restricted to high-cardinality numeric features (nunique > 10) to preserve the integrity of binary and ordinal indicators, while explicitly excluding DTI-related features.

2.6 One-Hot Encoding & Final Matrix Construction

We used a `ColumnTransformer` to convert the cleaned feature matrix into a modeling-ready design matrix. Key steps included binarizing `state_code` and treating low-cardinality numeric-coded fields as categorical. We applied `OneHotEncoder` to handle unseen categories and prevent collinearity, storing the final `X_train_ohe` and `X_test_ohe` as sparse matrices for memory efficiency.

3. Feature Signal Analysis (Target Correlations)

This stage quantifies which features are most associated with approval outcomes while preserving strict train-only evaluation.

All statistics are computed on finalized training artifacts from the pipeline (`X_train_imputed` with `y_train`) to ensure consistency and avoid leakage from test data.

- Train size: **111,992 rows**
- Predictors analyzed: **53** (`40` numeric, `13` categorical)
- Numeric association: **point-biserial correlation** (with Spearman as a robustness check)
- Categorical association: **Cramer's V** and **mutual information**
- Redundancy diagnostic: numeric **Spearman** correlation (`|rho| >= 0.8`)

3.1 Key Signal Features

The numeric ranking (see `fig12_numeric_signal_top15.png`) is led by tract/valuation-income structure variables, including `tract_one_to_four_family_homes` , `tract_owner_occupied_units` , `tract_population` , `loan_to_value_ratio` , and `lien_status` , with additional signal from `construction_method` and income-related tract indicators.

Categorical ranking results are summarized in `fig13_categorical_signal_top15.png` and represent the current category-level effects used for interpretation.

These rankings should be used for presentation and modeling decisions.

3.2 Multicollinearity / Redundancy Findings

We identified **13 highly correlated numeric pairs** (`|Spearman| >= 0.8`). Most of these relationships occur among co-applicant observation variables, applicant observation variables, and mortgage-structure flags.

This indicates meaningful redundancy in the raw feature space. Practically, the modeling phase can either prune near-duplicate variables for interpretability or retain all encoded information and reduce dimensionality using latent-factor methods.

4. Unsupervised Learning (TruncatedSVD)

Given the finalized one-hot encoding pipeline outputs a sparse design matrix, `TruncatedSVD` is the appropriate dimensionality-reduction method for this project.

Unlike dense PCA, SVD operates directly on sparse matrices, preserves computational efficiency, and avoids unnecessary densification risk.

4.1 Method and Controls

The unsupervised step is fit on `X_train_ohe` only, with explicit alignment checks against `y_train`. This keeps the full workflow consistent with train-only preprocessing and prevents leakage from test data.

For interpretability, component loadings are mapped back to encoded feature names (`feat_names`, or encoder-derived names when needed).

4.2 Results and Practical Use

SVD component selection is based on cumulative explained variance thresholds (90% / 95%) over a practical component range. The resulting latent features (`X_train_svd`, and `X_test_svd` when available) provide a compact representation for downstream modeling.

Top-loading summaries identify which encoded variables drive each latent component, supporting interpretability despite dimensionality reduction.

4.3 Modeling Recommendation

Use SVD-reduced features as the primary compact representation and compare them against original feature-selected baselines on validation AUC, F1, PR-AUC, and calibration.

In practice, feature-signal rankings remain the interpretability anchor, while SVD helps reduce redundancy and stabilize model training in the sparse high-dimensional space.